01211373 Machine Learning and Programming for Industry

Lecture 10 Supplement 1

Modeling sequential data with recurrent networks

RNNs · backpropagation through time · vanishing gradients · LSTM and GRU · generation

Running example: a CNC turning cell

Five channels at 50 Hz for every part — and a finished-diameter class that depends on an event at sample 25 AND a load level at sample 100

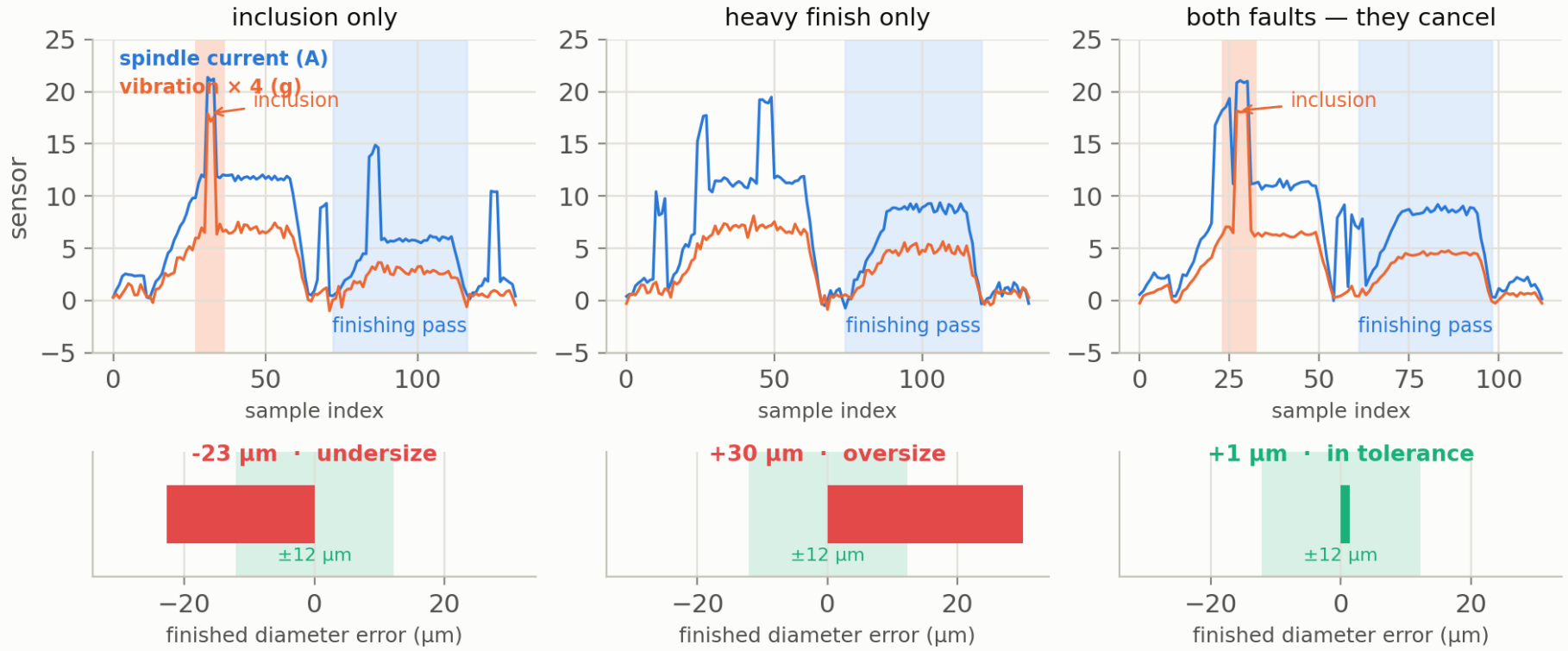
Companion notebook: mlpi26_Lect10_nbsup1.ipynb

Upper-undergraduate engineering · every number on these slides is computed in that notebook

THE TASK

Two faults, 100 samples apart, that cancel

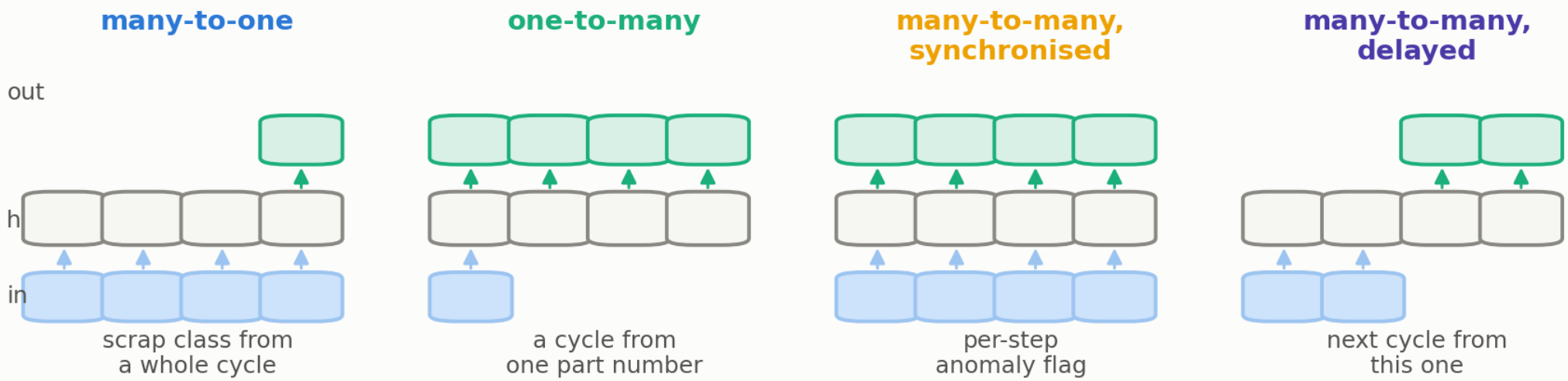
- A hard inclusion early in roughing chips the edge and leaves the part undersize; a heavy finishing pass deflects the tool and leaves it oversize. Either alone is scrap.
- With both, the two errors cancel to within the ±12 µm band and the part is good — so the class depends on both ends of the cycle and on neither alone.



800 cycles to train on, 110–150 samples each. Error budgets whose terms cancel are ordinary; what makes this one a teaching example is that the terms are 100 samples apart.

Pick the shape before you pick the model

- The relationship between input and output decides where you read the network's output from, and what the loss is computed over. Get this wrong and nothing else matters.

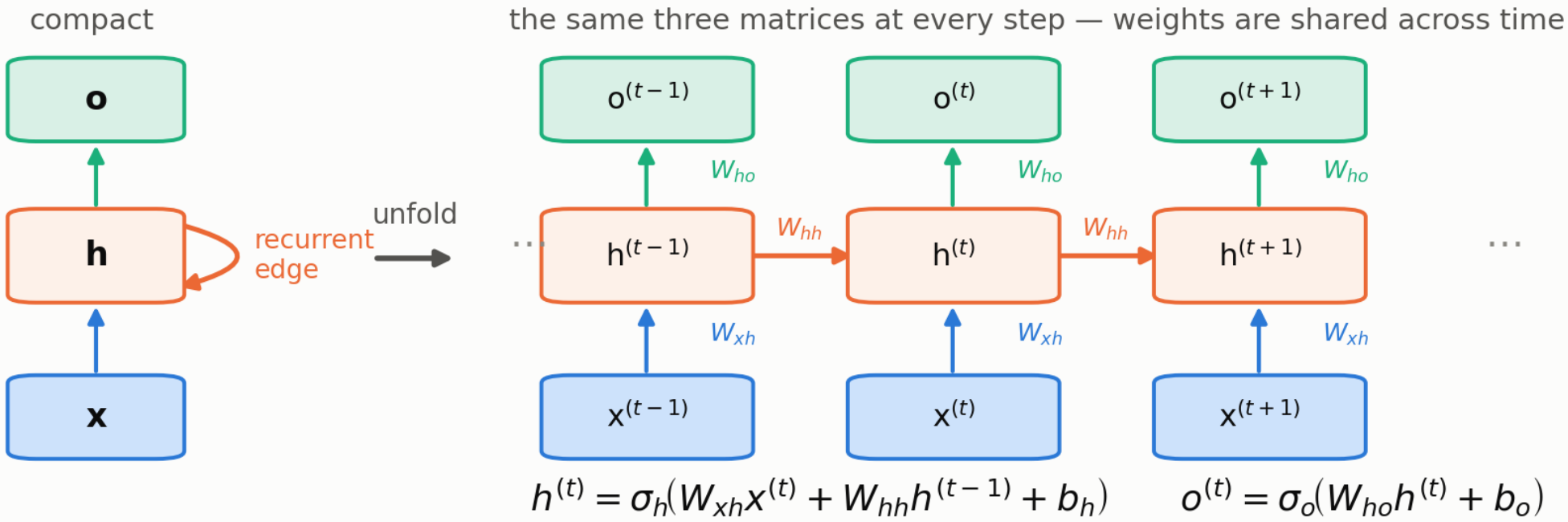



```
out, (h, c) = lstm(packed)       # out: every step   h: the last step

many-to-one              ->  h[-1]                        (project one)
many-to-many, aligned    ->  out, loss masked by length   (per-step flag)
many-to-many, delayed    ->  out, targets shifted by one  (project two)
```

An RNN is one extra edge

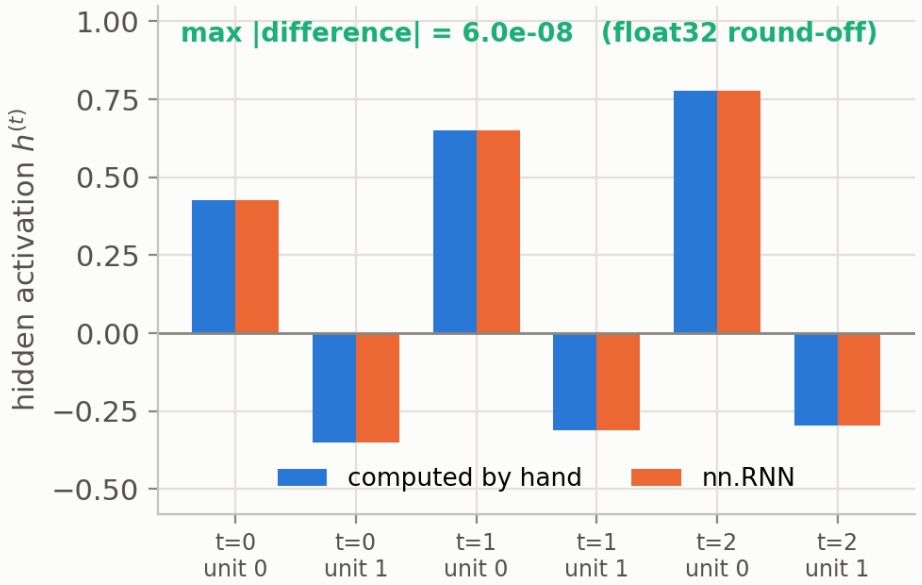
- The hidden layer receives its own previous value as well as the current input. Unfold that loop and you have a very deep network whose layers all share the same three matrices.



W_xh, W_hh and W_ho do not depend on t. That is parameter sharing across time, exactly as a CNN shares a kernel across space — and it is why one model handles a cycle of any length.

The forward pass, written out by hand

- Three time steps, two hidden units, computed with matmul and tanh and compared with nn.RNN — the two agree to 6.0e-8, which is float32 round-off.



what nn.RNN(4, 2) actually holds

`weight_ih_l0` `(2, 4)` W_{xh}: input → hidden

`weight_hh_l0` `(2, 2)` W_{hh}: the recurrent edge

`bias_ih_l0` `(2,)` b_{xh}

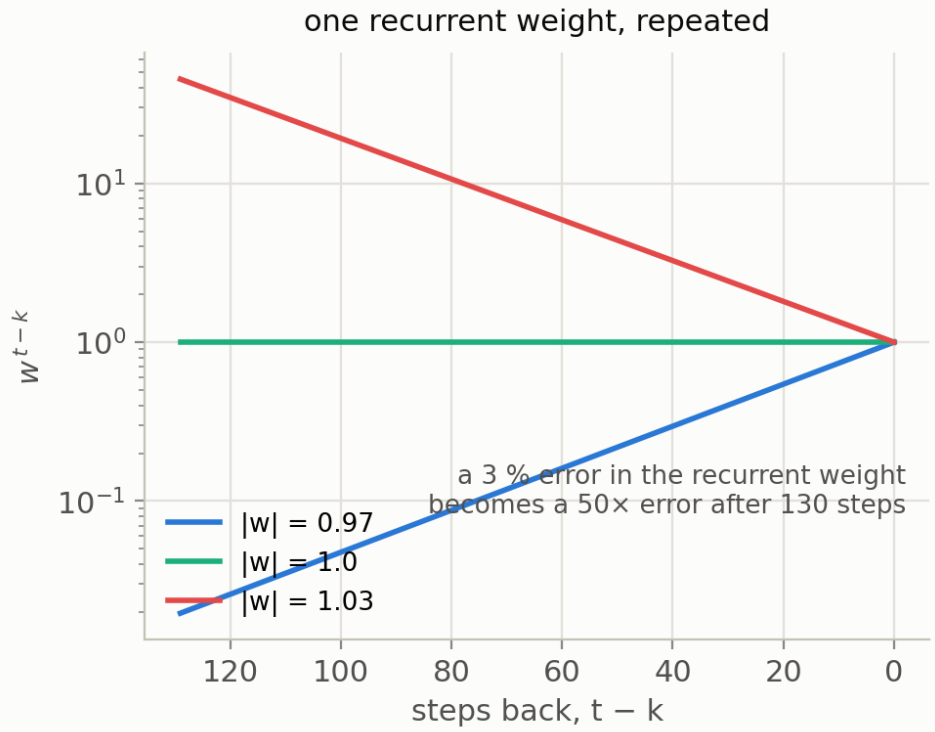
`bias_hh_l0` `(2,)` b_{hh} (PyTorch keeps two biases)

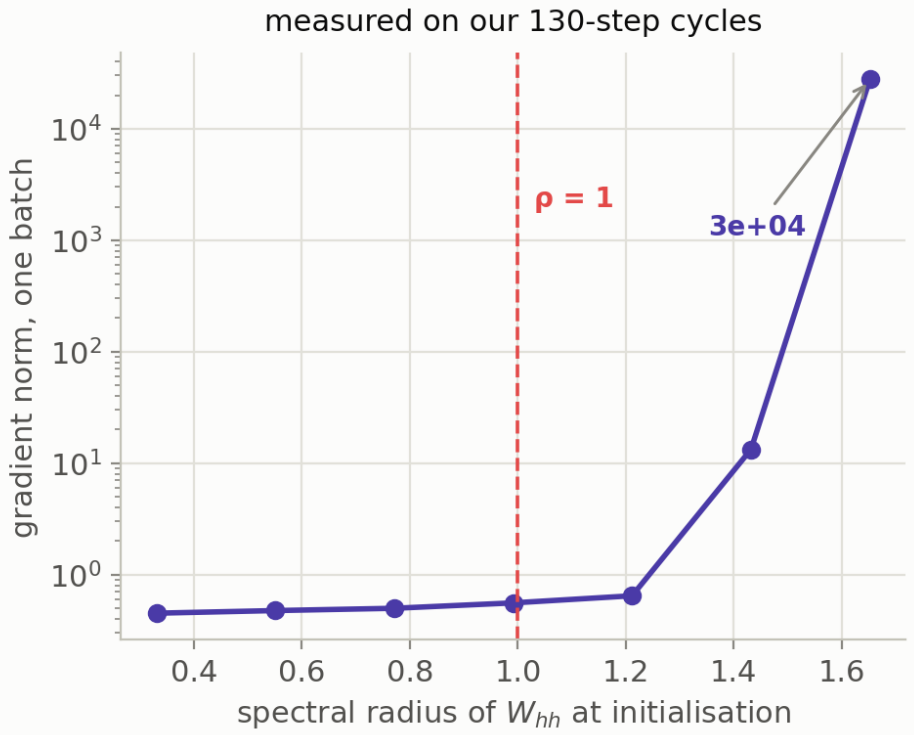
batch_first=True → input is (batch, time, features)

```
prev_h = torch.zeros((1, 2))
for t in range(3):
    ht = xt @ w_xh.T + b_xh                        # the input path
    ot = torch.tanh(ht + prev_h @ w_hh.T + b_hh) # + the recurrent path
    prev_h = ot                                    # the entire memory
```

BPTT multiplies Jacobians, and products misbehave

- The gradient reaching step k is a product of t − k Jacobians. For one unit that is w raised to the power t − k; for a matrix it is governed by the spectral radius ρ(W_hh).



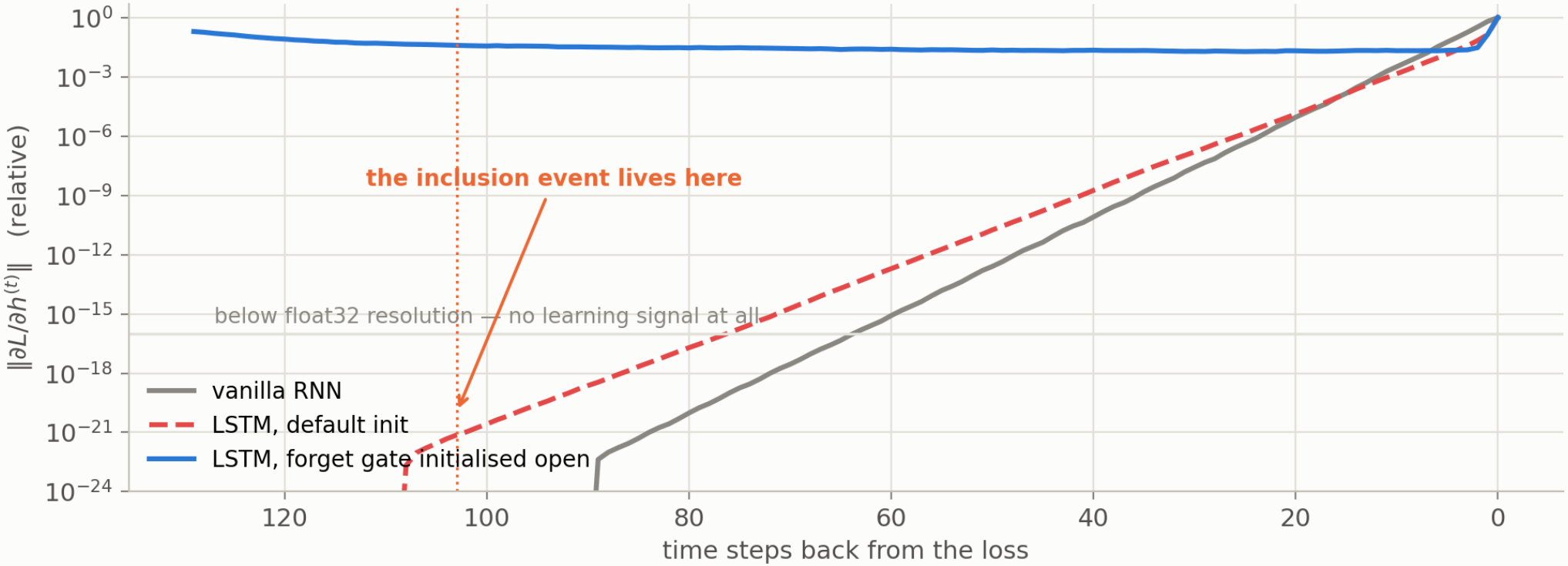


Measured on our cycles: the gradient norm is flat while ρ < 1.2, then jumps to 3e+4 at ρ = 1.65. Five orders of magnitude, from scaling one matrix.

MEASURE IT, DO NOT ASSUME

How far back does the gradient actually reach?

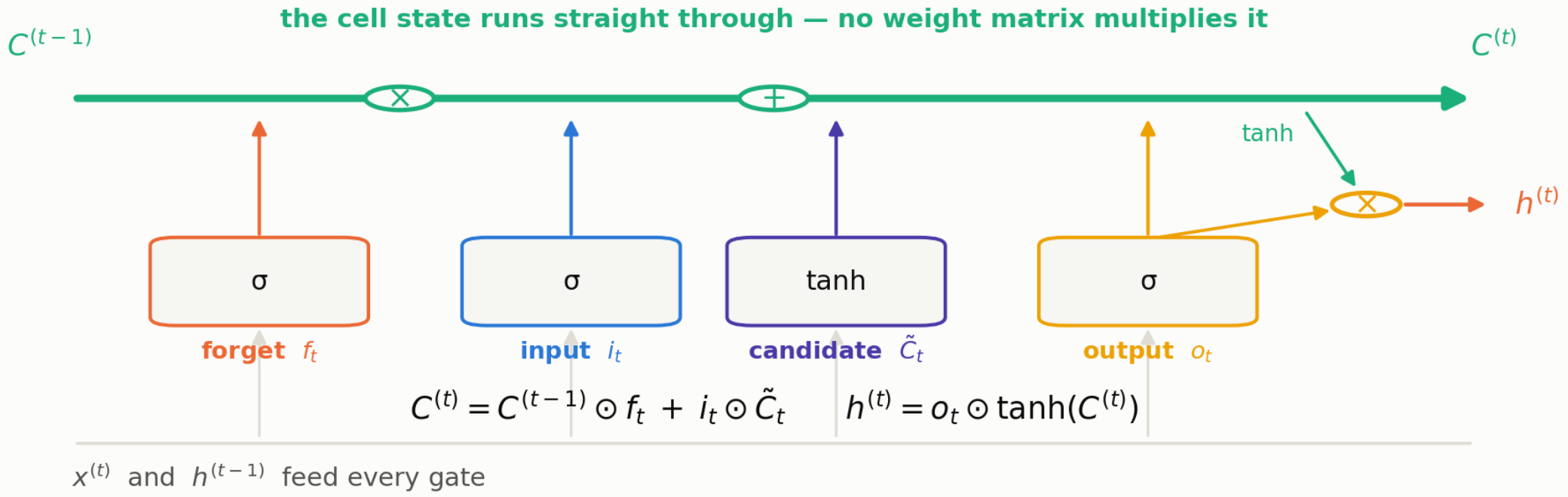
- Run an untrained cell for 130 steps, put a loss on the last step only, and record the gradient norm at every earlier step. It costs seconds and it is a property of the architecture, not the data.



100 steps back: vanilla RNN 0e+0 — exactly zero in float32 — and a default-initialised LSTM 4e-21. The evidence sits 103 steps back.

The LSTM adds a path with no matrix on it

- A second state, the cell state, runs from step to step through one element-wise multiply and one element-wise add. The gradient along it is multiplied by the forget gate, not by a Jacobian.

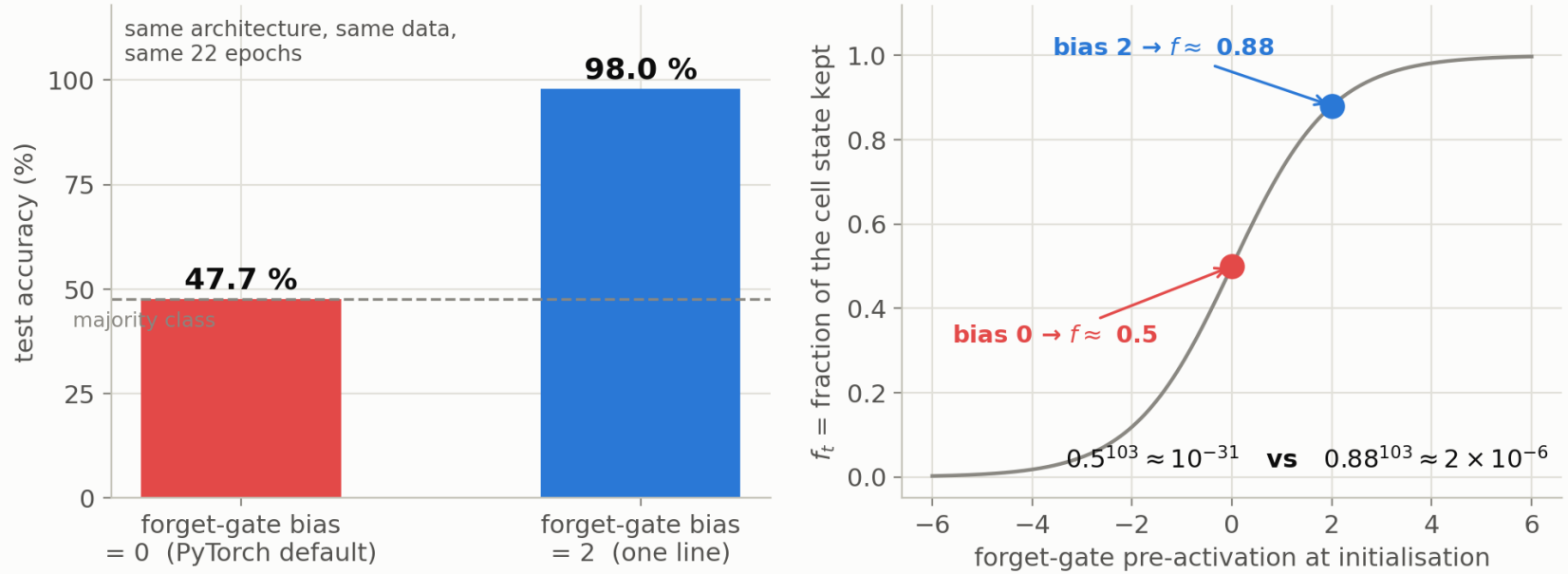


Forget gate: what to keep. Input gate and candidate: what to add. Output gate: what to expose. The forget gate was not in the 1997 cell — Gers et al. added it in 2000 because the state grew without bound.

THE ONE-LINE FIX

Initialise the forget gate open

- At initialisation every bias is near zero, so the forget gate sits at σ(0) = 0.5 — the cell discards half its memory at every single step. Over 103 steps that is a factor of 10^{-31}.

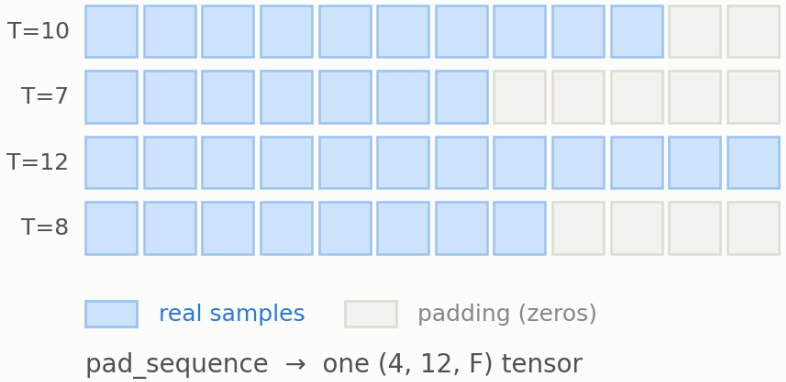



```
h = rnn.hidden_size                    # LSTM biases: [input, forget, cell, output]
for name, p in rnn.named_parameters():      # GRU biases: [reset, update, new]
    if name.startswith('bias'):
        p.data[h:2*h].fill_(2.0)       # the same slice works for both
```

Ragged batches: pad, then pack

- Our cycles run 110–150 samples, so a batch has to be padded to the longest member. Padding alone silently breaks the read-out.

four cycles in a batch, different durations



pad_sequence → one (4, 12, F) tensor

pack_padded_sequence tells the RNN where each cycle ends

Without packing, the recurrent layer keeps stepping through the padding, and the final hidden state you read out is the state after 12 steps of zeros — not the end of the cycle.

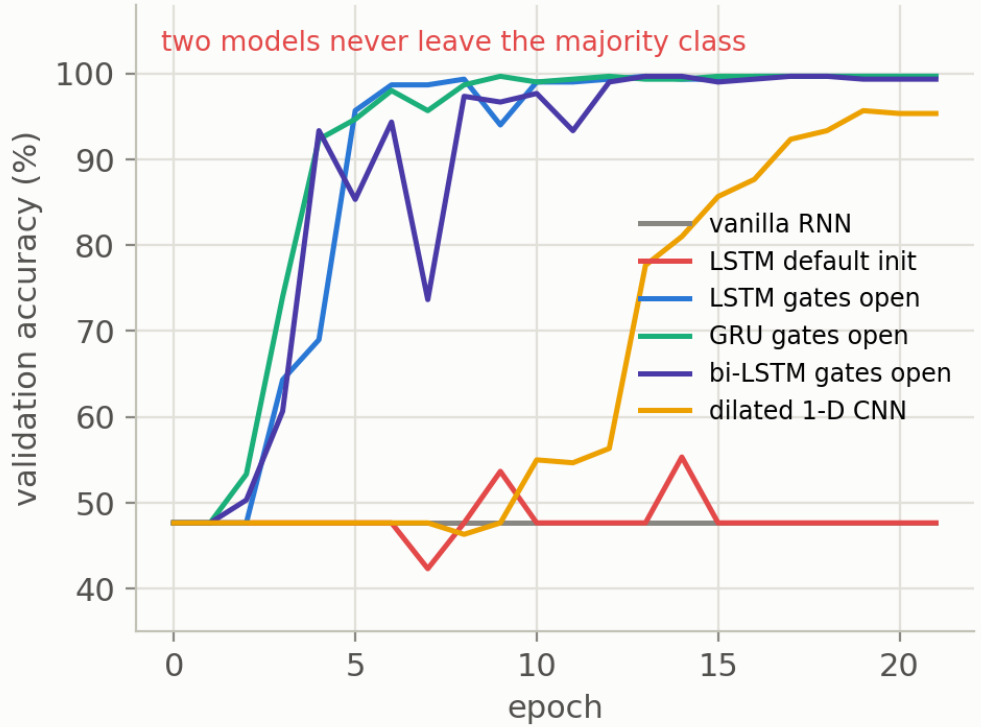
With packing, hidden[-1] is the state at the true last sample of every sequence in the batch.

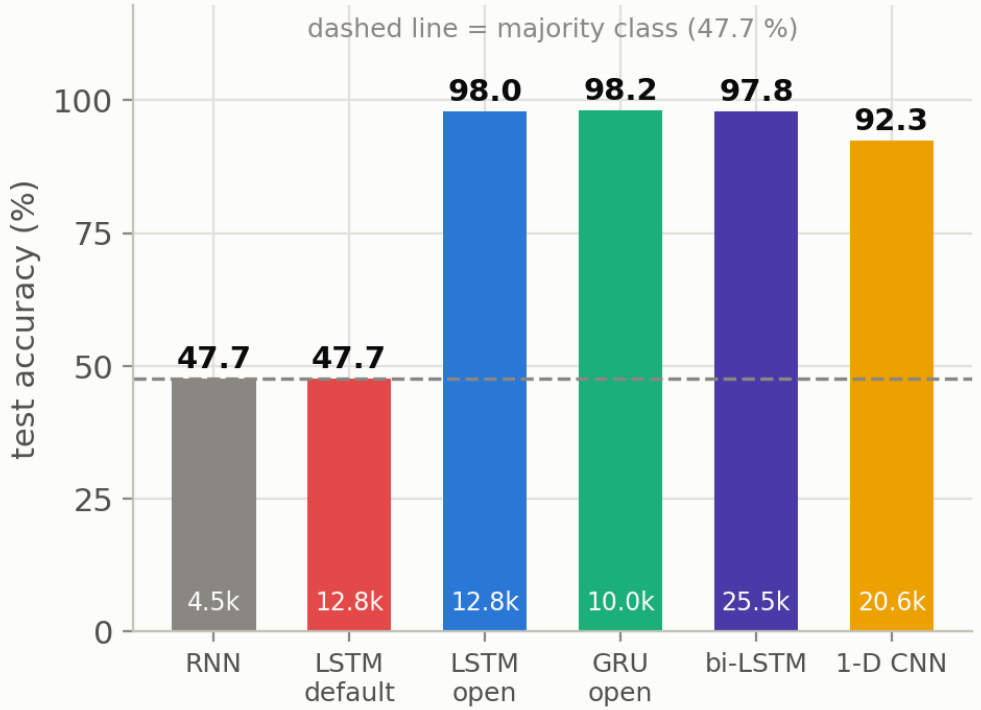
```
# WRONG -- hidden[-1] is the state after T_max steps, padding included
out, (h, c) = lstm(padded)

# RIGHT
packed = nn.utils.rnn.pack_padded_sequence(padded, lengths.cpu(),
                         batch_first=True, enforce_sorted=False)
out, (h, c) = lstm(packed)    # h[-1] = state at each true last sample
```

Six models, one dataset, 22 epochs each

- Vanilla RNN 47.7 %, LSTM with default init 47.7 % — both stuck at the majority class. Gates open: LSTM 98.0 %, GRU 98.2 %, bidirectional 97.8 %, dilated 1-D CNN 92.3 %.



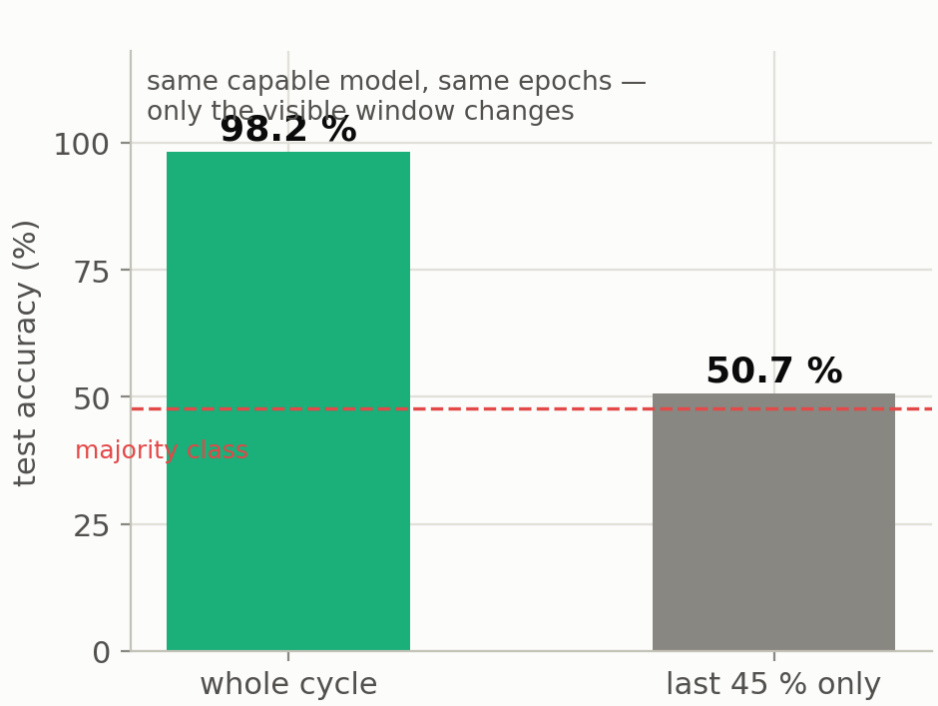


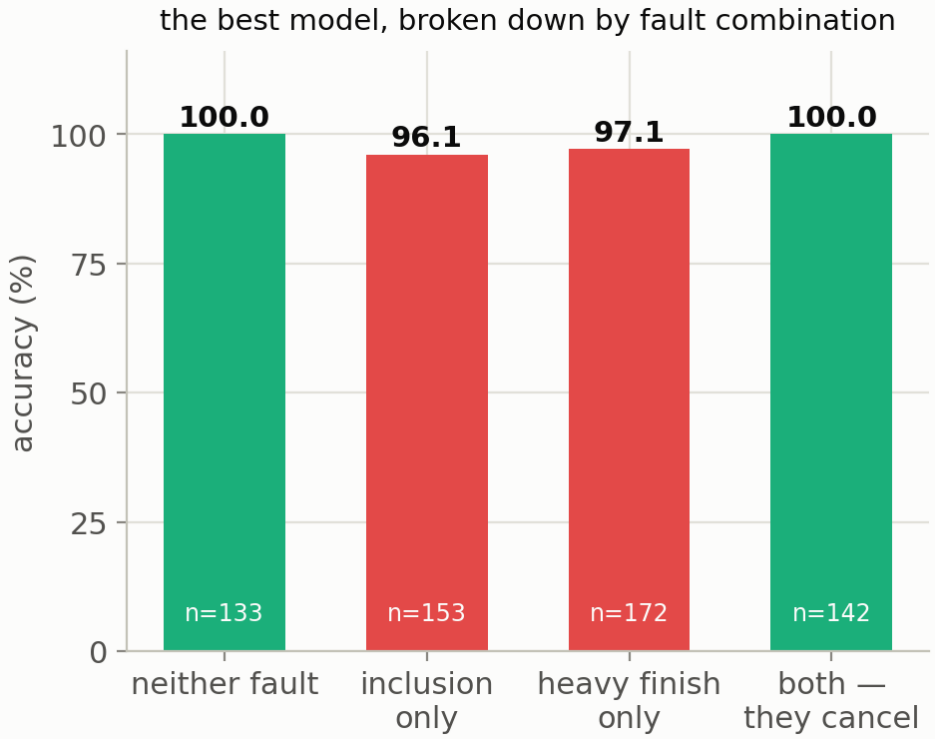
The GRU wins on 10,039 parameters — fewer than the LSTM and 2.5× fewer than the bidirectional model, which buys nothing here.

THE ABLATION

Prove the information is where you think

- Train the same capable model on only the last 45 % of every cycle. It lands at 50.7 % — chance — so the 98 % really does come from carrying the early event forward.

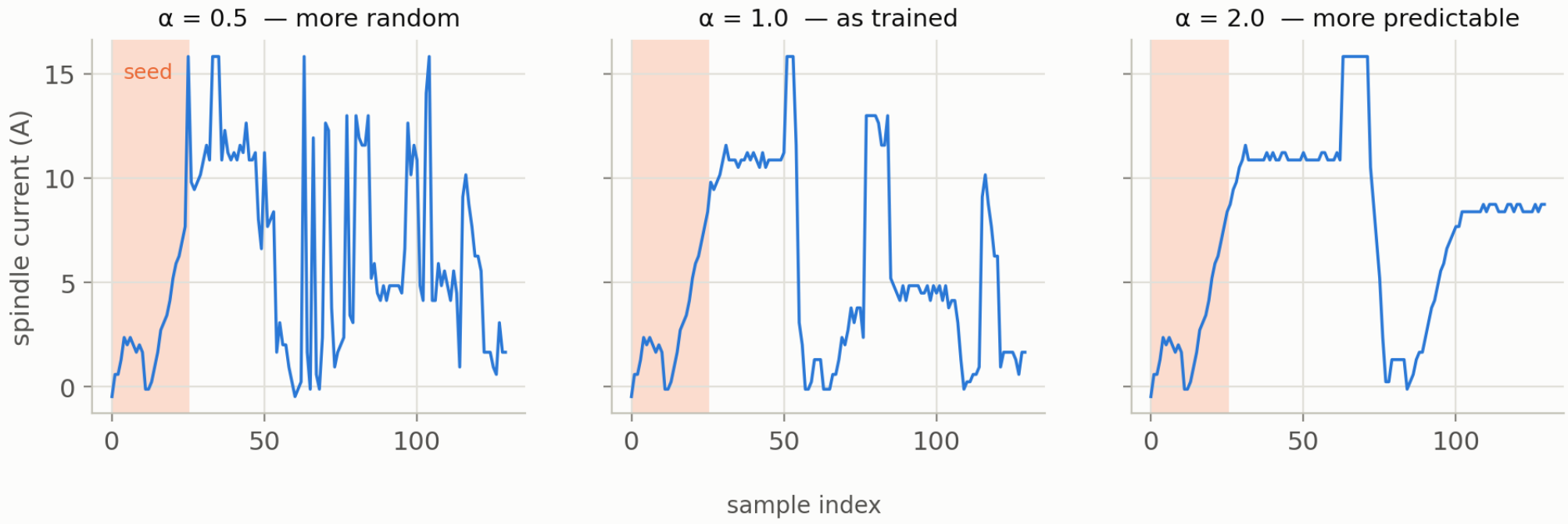




Run the experiment that could falsify your claim. This one takes eleven seconds and it is the difference between a measurement and a story.

Many-to-many: generating a cycle

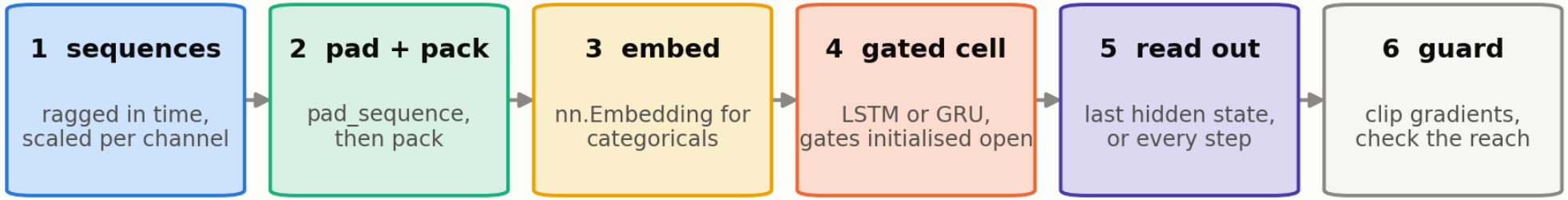
- Discretise the spindle current into 48 bins and next-sample prediction becomes next-token prediction — the book's character model, with load levels instead of characters. Sample from the logits; scale them by α to trade novelty against correctness.



the same model and the same seed samples — only the logit scale α changes

Everything so far squeezes the past through one fixed-width vector. Attention removes that bottleneck — the read-out looks at every step directly, so the gradient path is one multiplication, not 103. That is chapter 16.

What to take away



1. Pick the shape first — many-to-one, one-to-many or many-to-many decides whether you read h[-1] or out.
2. The same three matrices are applied at every step. That is parameter sharing across time, and it is the whole idea.
3. BPTT multiplies t − k Jacobians, so gradients vanish below ρ(W_hh) = 1 and explode above it. Nothing is subtle about it.
4. Measure the gradient reach before choosing an architecture. It takes seconds; assuming takes a week.
5. Initialise the forget gate open for anything over ~50 steps. One line was worth 50 accuracy points here.
6. Pad, then pack. Without packing, h[-1] is the state after the padding, and nothing warns you.
7. Run the ablation that could falsify your claim — hide the evidence and check the model breaks.

Generated by Claude and customized by

dew.ninja 2026

Notebook: mlpi26_lect10_nbsup1.ipynb · 8 exercises · Raschka, Liu & Mirjalili ch. 15 · Hochreiter & Schmidhuber 1997 · Gers et al. 2000 · Pascanu et al. 2013 · Jozefowicz et al. 2015